SQL Query Optimization

Chapter 05 - Sub-Chapter 03

SQL Databases Series

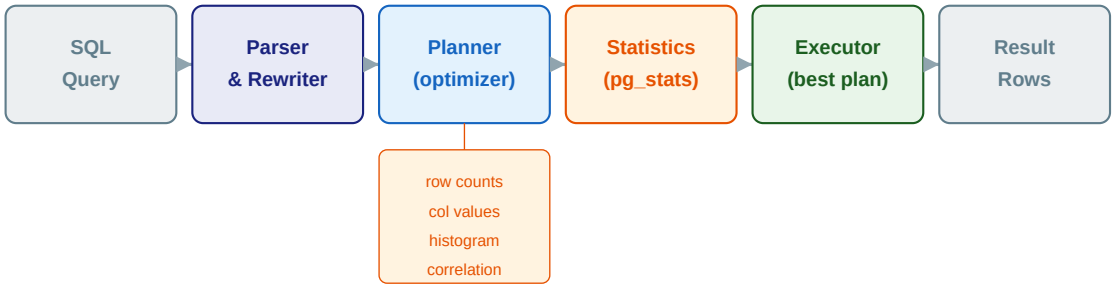
EXPLAIN ANALYZE	Read query plans, identify bottlenecks
Slow Query Patterns	Type casts, functions, SELECT *, missing WHERE
N+1 Query Problem	Detection, JOIN FETCH, @EntityGraph
Pagination & Statistics	Keyset vs OFFSET, pg_stat_statements

1 EXPLAIN and EXPLAIN ANALYZE

See exactly what PostgreSQL is doing

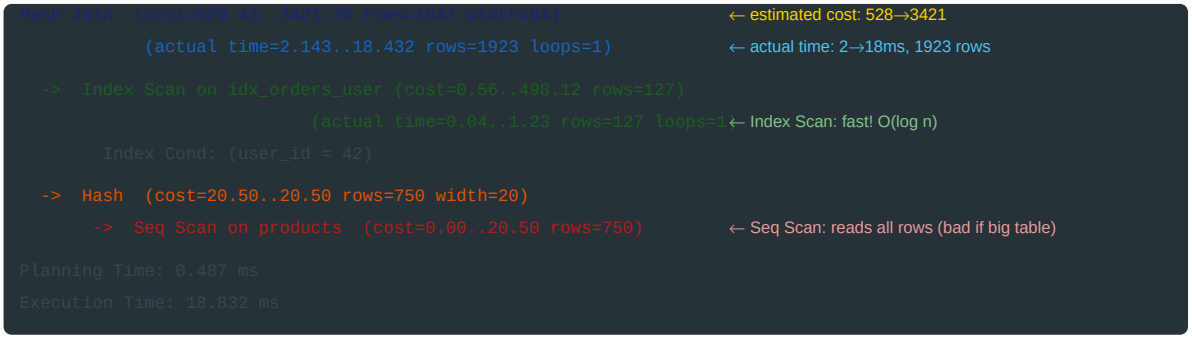
You cannot optimize what you cannot measure. **EXPLAIN** shows the query plan PostgreSQL will use — like seeing a map of the route before you drive. **EXPLAIN ANALYZE** actually executes the query and shows what really happened — plan vs. reality.

How PostgreSQL Chooses a Query Plan



Planner generates all possible plans (Seq Scan, Index Scan, Hash Join, etc.) and picks the lowest estimated cost

Reading EXPLAIN ANALYZE Output



Key: rows estimate ≈ actual rows = good stats | big gap = stale stats, run ANALYZE

Using EXPLAIN effectively:

```
-- EXPLAIN: shows estimated plan (no execution)
EXPLAIN SELECT * FROM orders WHERE user_id = 42;

-- EXPLAIN ANALYZE: executes and shows actual vs estimated
EXPLAIN ANALYZE SELECT * FROM orders WHERE user_id = 42;

-- EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT): full detail including cache hits
EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
SELECT o.order_id, u.name, SUM(oi.unit_price * oi.qty) AS total
FROM orders o
JOIN users u       ON o.user_id  = u.user_id
JOIN order_items oi ON o.order_id = oi.order_id
WHERE o.created_at > NOW() - INTERVAL '7 days'
GROUP BY o.order_id, u.name;

-- Key things to look for in the output:
-- 1. "Seq Scan" on a large table -- needs an index
-- 2. "rows=1000 actual rows=50000" -- stale statistics, run ANALYZE
-- 3. "Hash Join" vs "Nested Loop" -- optimizer's join strategy choice
```

```
-- 4. "Buffers: shared hit=X read=Y" -- X cached, Y from disk
-- 5. High "Planning Time" -- query is too complex, simplify
```

2 Common Slow Query Patterns

What to fix and how

Pattern 1: Implicit type cast breaks index use

```
-- Table: orders.user_id is INTEGER
-- Bug: passing VARCHAR breaks index use!
SELECT * FROM orders WHERE user_id = '42';   -- cast mismatch!
-- Plan: Seq Scan (implicit cast prevents index use)

-- Fix: match types exactly
SELECT * FROM orders WHERE user_id = 42;     -- integer literal
-- Plan: Index Scan using idx_orders_user_id  ✓

-- JDBC / Spring: always use typed parameters
// BAD:
jdbcTemplate.query("SELECT * FROM orders WHERE user_id = ?", "42");
// GOOD:
jdbcTemplate.query("SELECT * FROM orders WHERE user_id = ?", 42L);

// Spring Data JPA: parameter types are inferred from method signature
List<Order> findByUserId(Long userId);  // correct — Long matches BIGINT
```

Pattern 2: Functions on indexed columns disable the index

```
-- BAD: function wraps the indexed column -- index not used
SELECT * FROM orders WHERE DATE(created_at) = '2024-01-15';
-- Plan: Seq Scan (DATE() function prevents index use on created_at)

SELECT * FROM users WHERE LOWER(email) = 'alice@example.com';
-- Plan: Seq Scan (LOWER() prevents index use on email)

-- Fix 1: rewrite to avoid the function on the column
SELECT * FROM orders
WHERE created_at >= '2024-01-15' AND created_at < '2024-01-16';
-- Plan: Index Scan on created_at  ✓

-- Fix 2: create an expression index (covered in Sub-Chapter 02)
CREATE INDEX idx_users_email_lower ON users(LOWER(email));
SELECT * FROM users WHERE LOWER(email) = 'alice@example.com';
-- Plan: Index Scan on idx_users_email_lower  ✓
```

Pattern 3: SELECT * fetches unnecessary data

```
-- BAD: fetches all columns including large TEXT/BYTEA/JSONB blobs
SELECT * FROM products WHERE category = 'electronics';
-- Transfers description (10KB), images (100KB) for each row -- slow!

-- GOOD: fetch only what the UI needs
SELECT product_id, name, price, thumbnail_url
FROM products WHERE category = 'electronics';

-- GOOD in Spring Data JPA -- projection interface
public interface ProductSummary {
    Long getProductId();
    String getName();
    BigDecimal getPrice();
}
List<ProductSummary> findByCategory(String category);  // only fetches 4 columns
```

```
-- GOOD with @Query + DTO projection
@Query("SELECT new com.example.dto.ProductDto(p.productId, p.name, p.price) " +
       "FROM Product p WHERE p.category = :category")
List<ProductDto> findSummaryByCategory(String category);
```

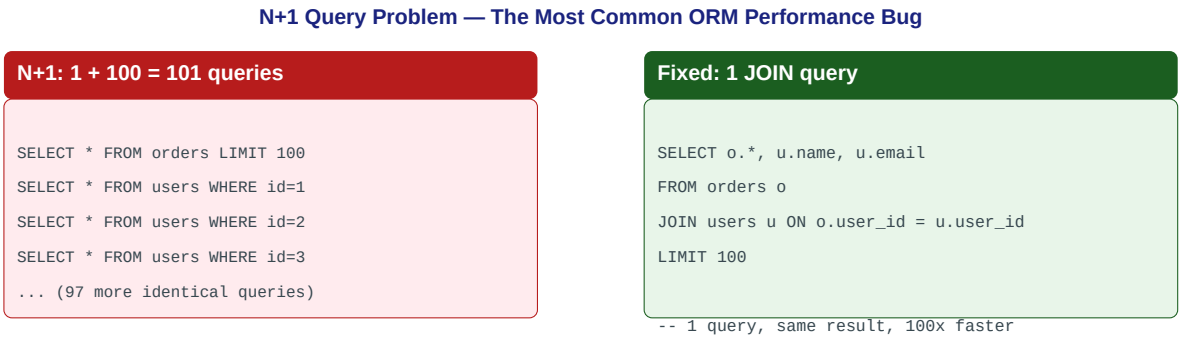
Pattern 4: Missing WHERE causes full table scan

```
-- BAD: updating status on ALL rows (accidental full scan + write)
UPDATE orders SET updated_at = NOW();   -- no WHERE!

-- BAD: COUNT(*) on filtered column without proper index
SELECT COUNT(*) FROM orders WHERE status = 'PENDING';
-- Plan: Seq Scan if no index on status

-- Fix: always verify WHERE clauses and ensure indexed columns
CREATE INDEX idx_orders_status ON orders(status) WHERE status IN ('PENDING','PROCESSING');
SELECT COUNT(*) FROM orders WHERE status = 'PENDING';
-- Plan: Index Only Scan -- count from index, no heap access
```

3 N+1 Query Problem

The silent killer of ORM-based applications

The N+1 problem is the most common performance bug in applications using JPA/Hibernate. You load a list of N orders, then for each order the ORM lazily loads the associated user — resulting in 1 + N database queries. With N=100, that is 101 round trips to the database.



Detecting N+1 in Spring Boot (enable SQL logging):

```
# application.yml -- enable SQL logging to spot N+1
logging:
  level:
    org.hibernate.SQL: DEBUG
    org.hibernate.orm.jdbc.bind: TRACE

# Or use p6spy / datasource-proxy for production-friendly logging
# If you see the same SELECT repeated with different parameter values: N+1!
```

Fix 1: JOIN FETCH in JPQL (for one-to-many):

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.stereotype.Repository;
import java.util.List;

@Repository
public interface OrderRepository extends JpaRepository<Order, Long> {

    // BAD: lazy loading triggers N+1
    List<Order> findByUserId(Long userId);

    // GOOD: JOIN FETCH loads user in one query
    @Query("SELECT o FROM Order o JOIN FETCH o.user WHERE o.userId = :userId")
    List<Order> findByUserIdWithUser(Long userId);

    // GOOD: fetch multiple associations in one shot
    @Query("SELECT DISTINCT o FROM Order o " +
           "JOIN FETCH o.user " +
           "JOIN FETCH o.items " +
           "WHERE o.userId = :userId")
    List<Order> findByUserIdWithDetails(Long userId);
}
```

Fix 2: @EntityGraph (declarative, cleaner):

```
import org.springframework.data.jpa.repository.EntityGraph;
import org.springframework.data.jpa.repository.JpaRepository;
import java.util.List;

@Repository
public interface OrderRepository extends JpaRepository<Order, Long> {

    // @EntityGraph specifies which associations to eagerly load
    @EntityGraph(attributePaths = {"user", "items", "items.product"})
    List<Order> findByUserId(Long userId);

    // For named entity graph defined on the entity class
    @EntityGraph("Order.withDetails")
    List<Order> findByStatus(String status);
}

// On the entity:
@Entity
@NamedEntityGraph(name = "Order.withDetails",
    attributeNodes = {
        @NamedAttributeNode("user"),
        @NamedAttributeNode(value = "items", subgraph = "items.product")
    },
    subgraphs = @NamedSubgraph(name = "items.product",
        attributeNodes = @NamedAttributeNode("product"))
)
public class Order { ... }
```

4 Pagination and Statistics

Keyset pagination, ANALYZE, pg_stat_statements

OFFSET pagination — the hidden performance killer:

OFFSET N skips the first N rows by reading and discarding them. Page 1 of 20 is instant. Page 100,000 of 20 scans 2,000,000 rows — increasingly slow.

```
-- BAD: offset pagination (degrades linearly with page number)
SELECT order_id, created_at, total
FROM orders
WHERE user_id = 42
ORDER BY created_at DESC
LIMIT 20 OFFSET 2000000;   -- reads and discards 2M rows!

-- GOOD: keyset (cursor) pagination -- always O(log n)
-- Page 1 (first page)
SELECT order_id, created_at, total
FROM orders
WHERE user_id = 42
ORDER BY created_at DESC, order_id DESC
LIMIT 20;

-- Page 2: pass last values from page 1 as cursor
-- (last_created_at = '2024-01-10', last_order_id = 12345)
SELECT order_id, created_at, total
FROM orders
WHERE user_id = 42
  AND (created_at, order_id) < ('2024-01-10', 12345)  -- cursor condition
ORDER BY created_at DESC, order_id DESC
LIMIT 20;
-- Index: (user_id, created_at DESC, order_id DESC) serves this perfectly!
```

pg_stat_statements — find the slowest queries in production:

```
-- Enable in postgresql.conf:
-- shared_preload_libraries = 'pg_stat_statements'
-- pg_stat_statements.track = all

-- Install the extension
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

-- Top 10 slowest queries by total execution time
SELECT
    LEFT(query, 100) AS query_preview,
    calls,
    ROUND(mean_exec_time::numeric, 2) AS avg_ms,
    ROUND(total_exec_time::numeric, 2) AS total_ms,
    ROUND(stddev_exec_time::numeric, 2) AS stddev_ms,
    rows / calls AS avg_rows
FROM pg_stat_statements
WHERE calls > 100              -- filter out one-off queries
ORDER BY total_exec_time DESC
LIMIT 10;
-- Focus optimization effort on high-total-time queries first!

-- Reset stats after each tuning session
SELECT pg_stat_statements_reset();
```

Key Rules

- EXPLAIN ANALYZE every query you optimize -- measure before and after, never guess.
- Avoid functions on indexed columns in WHERE (DATE(), LOWER(), etc.) -- rewrite or expression-index.
- Detect N+1 with SQL logging -- fix with JOIN FETCH or @EntityGraph in JPA.
- Use keyset pagination for large datasets -- OFFSET degrades at scale.
- Enable pg_stat_statements -- it shows you exactly which queries need attention in production.
- Run ANALYZE after large data loads -- stale statistics cause the planner to pick bad plans.